

Certified Control of the Full 600-Cell

Orbit structure, constructive algorithms, and a validated native workbench

C600 Studio project

Technical report, September 2026

Abstract

The full-cut 600-cell is a four-dimensional twisty puzzle with 177,120 surface pieces and 259,800 sticker slots. This report reconstructs the mathematical and engineering development documented in a 52-page design transcript, its accompanying 51-page algorithm reference, and the subsequent C600 Studio 0.2.4 validation. The central construction combines an exact orbit census, two-buffer setup trees, orientation-aware three-cycles, and a dependency order that retains every collateral move. An additional bounded computation verifies nine independent single-sticker controllers and establishes a conservative lower bound exceeding 10^{151851} face-color states. This exceeds an unconstrained upper bound for the solved 4^7 hypercube by more than 86,000 decimal orders of magnitude, without implying a universal ranking of human difficulty. Three archived synthetic solutions each represent approximately 353 million primitive turns; their compact execution is distinguished from manual solving and native animation. The native workbench subsequently passed full-state, rendering, interaction, recovery, and packaging checks on Windows. Its measured integrated-GPU performance supports adaptive motion, while a minimum GPU for continuous full-detail rotation at 1920×1080 and 30 FPS remains unestablished. The resulting contribution is a reproducible computational framework and a usable research instrument, rather than a claimed human solving record.

Keywords: 600-cell; permutation puzzles; computational group theory; nonabelian orientation; certified macros; reproducibility; human-computer interaction.

1 Scale and significance

The number 600 describes the tetrahedral cells of the underlying regular polytope, not the number of pieces requiring control. At the retained cuts of ± 0.968 , the full puzzle contains 433 sticker regions per cell, 177,120 physical surface pieces, and 35 moving position orbits. Its 600 fixed single-sticker centers account for the difference between 177,120 total pieces and 176,520 moving pieces. Andrey Astrelin's original announcement already identified the 259,800-sticker scale. The present construction supplies the finer orbit-level machinery needed to work with that scale [1, 2, 3].

The increase is particularly substantial relative to the simplified 600-cell. That subsystem retains only three moving families, together with the fixed centers: 2,640 total pieces and 9,000 stickers. Restoring the omitted pieces adds 174,480 pieces in 32 moving orbits. The ratios are approximately 67.09 in total pieces and 28.87 in stickers. A successful simplified solve therefore provides a valuable structural starting point while leaving most of the full puzzle to be controlled [1, 2, 4].

The strongest defensible statement about orders of magnitude concerns configurations, rather than a universal measure of difficulty. Section 4 constructs at least 10^{151851} distinct face-color states using only nine of the full puzzle's orbits. For comparison, a 4^7 hypercube has 57,344 stickers and 14 face colors. Even allowing every balanced color assignment, including illegal ones, gives

fewer than 10^{65696} states. The conservative ratio is therefore greater than 10^{86155} . The 4^7 is a meaningful high-dimensional comparator because a completed solve is recorded in the original MC7D Hall of Fame [5].

This comparison does not make the full 600-cell the largest solved-or-unsolved puzzle under every definition. The same Hall of Fame includes the 3^7 , whose 10,206 stickers provide another dimensional benchmark, but a documented team solve of a 256^3 cube involves 393,216 stickers, more than the full 600-cell [5, 6]. Dimensions, sticker counts, reachable configurations, recognition demands, and solution lengths are separate quantities. The number of colors also matters: throughout this report, a color means a distinct solved cell or face symbol, not a claim that a display can make hundreds of RGB shades equally easy to recognize.

Table 1: Scale of the named puzzle comparisons and the evidence used for each status.

Puzzle or model	Sticker slots	Status used for comparison	Interpretation
Simplified 600-cell	9,000	Published completed solve	Three moving families; same underlying cell geometry
3^7 hypercube	10,206	Original MC7D solve records	High-dimensional benchmark
4^7 hypercube	57,344	Original MC7D solve record	Named comparator for the color-state upper bound
Full 600-cell	259,800	Constructive model solutions and validated workbench	Thirty-five moving orbits; no human record asserted here
256^3 cube	393,216	Published participant account of a team solve	Counterexample to a universal sticker-count ranking

Solving this puzzle is useful as a demanding test of constructive control and verifiable computation. A workable method must coordinate many move orbits, distinguish piece identity from appearance, represent noncommuting orientation changes, and preserve already completed work. It also exposes an engineering problem with wider relevance: a system must execute compressed operations quickly while retaining an independently checkable account of everything those operations do. The scientific value lies in explicit constructions, failure analysis, reproducible data, and precise evidence boundaries. Large configuration counts alone neither prove a lower bound on human effort nor establish a new complexity class.

2 Documentary basis and antecedents

2.1 Primary corpus and method

The principal documentary source is *600 Cell Solving Strategy*, a 52-page exported design transcript [1]. Its first 18 pages develop the full model and constructive solving method; pages 19–24 examine a simplified solve; pages 25–31 explain the algorithm reference, replay records, and workload; pages 32–52 develop and debug the graphical workbench. The accompanying *Full 600-Cell: 35-Orbit Algorithms* supplies one algorithm card for every moving orbit, exact words, orientation procedures, and a workload analysis [2].

The transcript is treated as a record of claims and design decisions, not as proof merely because an assertion appears in it. The analysis separates mathematical arguments, finite computational certificates, historical execution reports, synthetic integration tests, and subsequent native Windows checks. A new audit verifies the nine controllers used in the configuration lower bound. The public companion preserves synthetic replay data and the original algorithm reference, while a source guide maps the report’s discussion back to transcript page ranges. The original conversation export, account-bearing links, personal sessions, and third-party solve histories are not included in the public research bundle.

2.2 Prior solving and software foundations

Magic Puzzle Ultimate, including its original puzzle simulator and renderer, is Andrey Astrelin’s work. C600 Studio is a modification built around that foundation. Its contribution consists of an authoritative full-state engine, certified solving tools, a journal, and adaptations around the inherited native viewport; it does not replace the original authorship claim [3, 7].

Nan Ma’s simplified 600-cell solve provides methodological background. His account solves faces, then edges, then corners, using the spherical projection’s surface as a buffer region. A piece travels across that surface to a point above its destination, is inserted inward to the appropriate layer, and is reoriented; the buffer surface is finished last. The completed simplified solve demonstrates that setup-conjugated local operations can make a very large four-dimensional puzzle manageable. It does not establish that the same words preserve the 32 additional moving orbits of the full model [4].

The `ivan216 MPUlt` and `mc7d-kb` projects provide software background: an accessible MPUlt source reference and a control model that distinguishes grips, twists, layers, recentering, and configurable keys. The latter is Ivan’s fork of Aegyo’s keybinding extension to Andrey’s original MC7D. These projects provide relevant comparisons for integration and interface choices. Neither is presented here as the source of the full 35-orbit construction, and the present modifications do not imply their authors’ endorsement [8, 9].

3 The retained model and its invariants

3.1 Stable addresses and legal actions

The construction is specific to the `600-cell-Full` profile, with cuts ± 0.968 , `FixedMask 2`, and no `Simplified` flag. Changing these definitions changes the problem. Geometry is reconstructed from golden-ratio coordinates with floating-point region identification; subsequent state transitions use exact integer permutations. This is not a claim of certified exact geometric clipping. A native bridge later checks the correspondence between this enumeration and the inherited MPUlt runtime [1, 2, 7].

Let $M(P)$ be the set of turning caps containing a physical piece P , and $H(P)$ its hosting cells. The rank and sticker count are

$$r(P) = |M(P)| - 1, \quad k(P) = |H(P)|. \quad (1)$$

A position is identified by its complete cap-membership signature $M(P)$, and a sticker slot by the pair $(M(P), h)$, where h belongs to $H(P)$. Rank is a useful descriptive attribute, but it is not a unique orbit identifier. The toolkit’s O00 through O34 labels are enumeration-specific names, not original MPUlt menu labels.

For each cell c , H_c and T_c denote a legal half-turn and third-turn of its cap. There are 1,200 positive generators across 600 cells. The integer encodings are $2c + 1$ and $2c + 2$, with negative integers denoting inverses. A state array L records the home identity currently occupying each slot. For a source-to-destination permutation g , the update rule is

$$L'[g(s)] = L[s], \quad s = 0, \dots, 259799. \quad (2)$$

Words execute chronologically from left to right. Accordingly, $[U, V]$ means $UVU^{-1}V^{-1}$, and the cycle $(a\ b\ c)$ sends a to b , b to c , and c to a . Explicit conventions are necessary because reversing composition conventions can reverse a setup, a cycle, or a replay certificate.

3.2 Census and local orientation

The 35 moving orbits contain 176,520 pieces. The 600 fixed centers are 600 singleton position orbits; counting them literally gives 635 position orbits, while grouping centers as one geometric family gives 36 families. The complete 35-row census and algorithm cards are retained in the accompanying reference rather than replaced with rank-based approximations [2].

Table 2: Local orientation groups of the 35 moving orbits.

Local orientation group	Moving orbits	Practical consequence
Trivial	The remaining 24 orbits	No independent local orientation variable; this includes some two-sticker families
C_2	O00, O03, O08, O11, O22, O25, O32	Two local orientations; separate even flip parity in each orbit
C_5	O17, O28	Five orientations; separate transported-frame twist sum modulo five
D_5 , order 10	O06	Rotation and reflection-type states; reflection parity is constrained
A_5 , order 60	O34	Nonabelian corner orientation; a single residual orientation can be legal

Pairs O04/O05, O12/O13, O18/O19, and O26/O27 share rank and sticker count while remaining distinct move orbits. Combining them in a filter is legitimate; identifying them as the same orbit is not. Similarly, twenty corner stickers do not imply a twist variable modulo twenty. The attainable orientation group of O34 has sixty elements and noncommuting multiplication.

The invariant audit records even positional parity separately in every moving orbit. An odd permutation in one orbit cannot be canceled by an odd permutation in another. For O06, a reflection-type element of D_5 exchanges two pairs of five stickers and is therefore even as a sticker permutation. Ordinary sticker-permutation sign consequently does not detect its reflection bit. Orientation must be analyzed in the correct local group [1, 2].

These statements concern legally tracked labelled states. Assigning arbitrary identities to visually indistinguishable pieces can manufacture an apparent parity defect. Exact identities should be reconstructed from legal history, rather than guessed from the displayed colors.

4 A conservative configuration lower bound

4.1 Construction and verification obligations

The full reachable group order is not needed to establish a substantial scale gap. Consider the nine single-sticker orbits

$$\mathcal{I} = \{\text{O01, O04, O05, O07, O18, O19, O27, O31, O33}\}. \quad (3)$$

Their sizes are 3,600 for O01, 2,400 for O33, and 7,200 for each of the other seven. The audit replays each raw seed over all 259,800 labelled slots and checks that its complete effect is precisely one three-cycle in the stated orbit, with all other slots fixed. It also checks the relocation to the designated buffer positions, independently rebuilds the guarded setup search, and verifies coverage of all $N_o - 2$ nonbuffer destinations. Every orbit in $\mathcal{I}$ has one sticker per piece and trivial local orientation.

Let the relocated pure controller be $(A\ B\ C)$. For every nonbuffer X , a legal setup S fixes A and B and takes C to X . The conjugate $S^{-1}(A\ B\ C)S$ is therefore $(A\ B\ X)$. These star three-cycles generate A_{N_o} , the alternating group on that orbit. As the nine families have disjoint support and each controller is pure on the complete puzzle, their alternating-group actions can be combined independently.

The solved state has 600 distinct cell-color symbols. Within each selected orbit, every symbol occurs $m_o = N_o/600$ times: six, four, or twelve. The number of color arrangements realized by A_{N_o} equals $N_o!/(m_o!)^{600}$. To see why no factor of two is required, note that m_o is at least two. Exchanging two pieces of the same color is an odd permutation that stabilizes the coloring. Any odd representative of a color arrangement can thus be replaced by an even representative with the same visible assignment.

4.2 Bound and comparison

It follows that the full model has at least

$$B = \frac{3600!}{(6!)^{600}} \frac{2400!}{(4!)^{600}} \left[\frac{7200!}{(12!)^{600}} \right]^7 \quad (4)$$

distinct face-color states, with every unselected orbit held solved. Numerically, $\log_{10} B$ is approximately 151851.164322. This is a lower bound derived from a specific verified subgroup, not an asserted exact order for the full puzzle group and not a count of individually labelled stickers.

A side-length-four seven-dimensional hypercube has $2dn^{d-1} = 57,344$ stickers. With fourteen colors and 4,096 copies of each color, the number of all balanced face-color assignments is

$$U = \frac{57344!}{(4096!)^{14}}, \quad \log_{10} U \approx 65695.470510. \quad (5)$$

Every legal 4^7 state is among these assignments, so U is an upper bound regardless of its further permutation and orientation restrictions. Fixed-frame counting is used for both puzzles. Conservative outward rounding gives $B > 10^{151851}$ and $U < 10^{65696}$, hence $B/U > 10^{86155}$.

This argument uses comparable face-color states on both sides; it does not compare a labelled upper bound for one puzzle against a colored lower bound for another. It is also intentionally limited to a named, documented solved high-dimensional comparator. A configuration-space gap does not by itself yield a move-count lower bound, a cognitive difficulty ratio, or a claim that no larger puzzle has been solved. The reproducible audit and numerical calculation accompany the report.

5 Constructive solving across all 35 orbits

5.1 Seeds, collateral, and two execution modes

For orbit o , the reference supplies a legal seed word W_o whose target-orbit effect is a three-piece cycle. Twelve raw seeds have no collateral; the remaining twenty-three also move other orbits. A filtered image of the target cycle cannot determine which case applies. For example, the transcript gives a short word that cycles pieces in both O24 and O27; hiding rank fourteen conceals the second action [1, pp. 7–8]; [2].

Two execution modes follow. A pure controller is constructed by applying a raw seed and then correcting all its collateral through previously constructed pure controllers. The measured dependency graph is acyclic, so these corrections form a finite directed acyclic graph of legal words.

Purity describes the complete net effect: intermediate primitive turns may disturb pieces that return home before the macro finishes.

The shorter production method instead retains raw seeds and solves each target orbit before the orbits that its seed disturbs. All collateral then falls within unfinished stages. This triangular method is the workload baseline used by the archived synthetic solutions. Executing only the visible three-cycle while discarding its other effects would implement a different, uncertified operation.

5.2 Star operations and guarded setups

Let R_o relocate a seed's three targets to the reference positions A , B , C , and let $S_{X,q}$ carry the reference at C to a selected destination X with attainable ordered sticker frame q , while fixing A and B . A positive star is

$$K_{o,X,q} = S_{X,q}^{-1} R_o^{-1} W_o R_o S_{X,q}. \quad (6)$$

Its target-orbit action is $(A\ B\ X)$; its inverse acts as $(A\ X\ B)$ and also counts as one star. Buffer locations are fixed during setup, but their occupants move when the star executes. This distinction prevents the mistaken assumption that a buffer permanently contains the same physical piece.

The primitive cost is

$$|K_{o,X,q}| = |W_o| + 2|R_o| + 2|S_{X,q}|. \quad (7)$$

For O33, the pure seed $[[[H_0, T_1], H_{15}], H_{56}], H_{21}]$ has a stored length of 46 primitives. Its relocation adds ten moves, making a reference star 56 moves long; a target setup of depth d gives $56 + 2d$. Exact forward and inverse words, without ellipses, are retained for all thirty-five seeds in the reference companion [2].

Setup searches forbid primitive caps belonging to $M(A) \cup M(B)$. Their states include both the destination and its ordered sticker frame. The retained trees cover $(N_o - 2)|\Omega_o|$ oriented nonbuffer states for every orbit. For O34 this is $(120 - 2) \times 60 = 7,080$ states. Thus the method specifies finite lookup paths, rather than leaving setup discovery as an informal instruction.

5.3 Placement and orientation

For an incorrect nonbuffer destination X , let $Y = \text{where}(X)$ be the current position of the piece whose home is X . If $Y = A$, use K_X^{-1} ; if $Y = B$, use K_X ; otherwise, when Y differs from A , B , and X , use $K_Y^{-1} K_X$. The latter has target-position action $(A\ Y\ X)$, inserting the required piece at X while preserving already completed nonbuffer positions. Frame choices are included in the fully specified stars.

Once every nonbuffer position is correct, an isolated swap of A and B would be odd. The separately checked even positional parity in that orbit excludes this residual. This is an orbit-specific argument; combining parity across different families would be incorrect.

Orientation is then controlled by comparing a canonical star with a star using another attainable frame:

$$P_X(q) = K_{X,q} K_X^{-1}. \quad (8)$$

This fixes positions and transfers inverse orientation changes between X and a buffer. A finite search among actual frames chooses the correction that restores X 's labelled stickers. After

orienting the nonbuffer positions and then the first buffer, only the final buffer can retain a defect. The generic final-buffer construction uses $[P_X(q), P_Y(r)]$ for distinct nonbuffer positions. Its eight constituent stars change only that buffer’s orientation on the target orbit.

For C_2 and C_5 families, the separate flip or twist-sum invariant forces the final residual to vanish. For D_5 , the residual lies in its rotational commutator subgroup; if R is a five-way rotation and F a reflection-type element, $[R^k, F] = R^{2k}$, and multiplication by two is invertible modulo five. For A_5 , the retained finite commutator table covers the required residuals. The argument uses actual group multiplication rather than a scalar approximation to corner orientation [1, pp. 9–11]; [2, pp. 4–6].

5.4 Preserving completed work

The safe raw-seed stage order is supplied with the data and follows the actual collateral dependencies, not monotonically increasing rank. After every macro, the solver applies its complete permutation. After every stage, it checks that all previously completed orbits remain solved. The final acceptance condition is $L[s] = s$ for every slot s .

Reduction, blockbuilding, and orbit-first solving differ primarily in preservation costs. Forgetting all but O00, O06, and O34 gives a legitimate simplified quotient. Solving that quotient leaves a kernel problem on the additional full-puzzle pieces; it does not justify treating omitted regions as permanently bandaged blocks. Graph-shell blockbuilding can use pure controllers to preserve completed pieces at macro boundaries, but does not promise that they remain motionless throughout each expanded word. Orbit-first solving with cell-local destination batching retains the tested dependency order while reducing navigation overhead. The retained evidence supports it as the practical baseline [1, pp. 12–13].

6 Workload, replay, and the meaning of a solve

6.1 Recorded computational results

The three synthetic full-model trials begin from 1,000-move scrambles with random seeds 600, 601, and 602. Their compact histories record every star and preserve the full collateral action. The archived totals are:

Table 3: Archived synthetic solutions. Historical solving times exclude preparation and Windows graphics.

Synthetic trial	Stars	Represented solution primitives	Historical headless solving time
Seed 600	374,178	352,698,206	8.08 s
Seed 601	373,948	352,080,470	7.86 s
Seed 602	374,428	352,839,050	7.24 s

The timing column belongs to the original container experiment and excludes preparation and Windows graphics. It is retained as historical evidence, not relabelled as a current hardware benchmark. The public companion contains the three synthetic histories and a standalone verifier. Its publication audit regenerates their scrambles and replays the compact full-state effects; the audit’s separate results identify exactly what was rerun.

A successful compact replay establishes that the recorded legal macro history restores the complete labelled model. Checking a saved color field alone would be weaker. Conversely, checking a huge native-format archive’s syntax, counters, nesting, and checksum would not constitute primitive-by-primitive geometric replay. The original transcript explicitly distinguishes these evidence levels [1, pp. 25–30].

The historical monolithic expansions are unsuitable as ordinary native-session files. Individual decompressed logs are approximately 2.9 GB, and the old 32-bit sequence allocator can require still more memory during growth. They are not included in the public companion. Compact certificates preserve the legal sequence without requiring such files to be loaded into the GUI. The 0.2.4 bounded log importer should likewise not be described as an importer for arbitrary multi-gigabyte histories.

The reference also gives a conservative method-specific bound of 405,945 stars and 385,797,152 represented primitives. The independent audit recomputes this bound from the retained orbit sizes, orientation groups, and maximum setup depths. It bounds this construction, not the shortest possible solution [2].

6.2 Human workload and assistance

A star is an algebraic unit, not a solved piece or an elementary hand action. Placement may require two stars, and an orientation transfer typically requires two. In the seed-600 trace, placement accounts for 345,833 stars, nonbuffer orientation for 28,302, first-buffer orientation for 27, and final-buffer commutators for 16. The mean represented cost is approximately 943 primitive turns per star [1, pp. 27–31]; [2].

For a scenario with t seconds of recognition, target/frame selection, checking, and application per star, the active-time model is

$$T_{\text{hours}} = \frac{374178 t}{3600}. \quad (9)$$

At 15–30 seconds per star this gives approximately 1,559–3,118 active hours, before learning, development, breaks, or recovery. These are throughput scenarios, not observations of human performance. At one primitive turn per second, the represented seed-600 solution would require about 97,972 hours of uninterrupted execution. Neither estimate is a lower bound on the puzzle’s optimal solution length.

The distinction between manual, macro-assisted, target-assisted, and automated solving must therefore be recorded explicitly. Assistance that chooses targets and frames changes the nature of an attempt even when every selected macro remains legal. Primitive counts, star counts, transactions, assistance labels, and active time should be retained separately. The framework demonstrates a complete computational route, while a human attempt remains an additional empirical undertaking.

7 From construction to a dependable workbench

7.1 State, certificates, and persistence

The transcript’s architectural decision is to make the 259,800-entry labelled array authoritative and treat rendering as a derived view. In 32-bit integers this array occupies 1,039,200 bytes, about 0.99 MiB. Geometry and projection, rather than the logical state alone, account for much of the resource burden.

A macro certificate identifies the model, legal recipe, full permutation digest, support by orbit, primitive/star counts, and before/after state hashes. These digests identify data and state transitions; they are not digital signatures proving authorship. The engine rechecks the preview revision and protected-orbit intersection at commit. It then applies the complete net effect as one durable transaction, retaining the legal witness for independent expansion or replay.

The journal preserves parent-linked history and checkpoints rather than deleting alternative branches. Reset and validated imports create recovery paths; undo and redo verify expected state transitions. A coherent native snapshot combines state, status, colors, and styles, avoiding a display assembled from different revisions. Filters and motion sampling change visibility only. Hidden pieces continue to receive every mechanical update and remain included in progress and solved-state checks [1, pp. 35–39]; [7].

7.2 Integration failures as evidence

The original transcript documents a real verification gap: source checks, layout arithmetic, and software-rendered browser tests did not establish that the native WinForms/DirectX path worked on Windows. A splitter initialization defect and a launcher assumption about matching parent/interpreter process IDs survived those earlier checks. The later local work also addressed renderer lifecycle and resize recovery, color/style synchronization, hidden-piece picking, and frame visibility [1, pp. 47–52]; [7].

The frame defect illustrates why a display policy must apply at every draw entry point. Guarding only the normal lifecycle path allowed inherited direct or animated redraws to bypass frame hiding. The repaired scene boundary applies the same policy to those redraws; a visible checkbox and View-menu item share a persisted setting. Visibility-aware picking rejects hidden pieces even when a frame is intentionally shown. These changes make filtering a reliable instrument for inspection while preserving the complete puzzle.

7.3 Subsequent native validation

The final 0.2.4 sources passed 68 comprehensive native checks and four fresh-process reopen checks using the actual MPUlt runtime, official installed Managed DirectX assemblies, native controls, viewport messages, and render-target pixels. Coverage includes twisting, 3D/4D rotation, colors, scrambling and undo/redo, filters and picking, keys, checkpoints, macros, buffer/insertion tools, reports, reset, log exchange, and controlled device-loss recovery. Three full-detail poses and a filtered selection view matched the comparison rendering path exactly [7].

Backend mechanics, independent generator maps, crash recovery, frame preferences, process lifecycle, frozen packaging, and a real isolated install/uninstall were checked separately. Normal executable startup preserves an existing session and does not run the developer WinForms fixture. These results close specific earlier native integration gaps. They do not retroactively turn the transcript’s original container timings into native measurements, and they do not establish that hundreds of millions of primitives were individually animated to completion.

8 Full-detail rotation and hardware requirements

8.1 Measured behavior

The final Intel HD Graphics 620 measurements distinguish exact filtered motion, adaptive dense-scene motion, and full-detail redraw. At a 931×604 viewport, optimized full-detail frames averaged 453.45 ms, compared with 626.57 ms through the comparison path, a 27.63% reduction in elapsed drawing time. The full-detail 95th percentile was 533.82 ms. The reciprocal of that mean is approximately 2.2 FPS; the eighteen fixed-pose redraw measurements are not a sustained rotation benchmark [7].

The approximately 58 FPS result is an adaptive-motion result; it is not full-detail rotation at 58 FPS. The complete mechanical state remains active in both modes. This distinction is necessary to evaluate the requested full-detail target rather than a visually reduced substitute.

Table 4: Measured interaction and redraw performance on the tested Intel HD Graphics 620 system.

Workload on the tested system	Measured result
Exact active-orbit / work-cell motion	56.63 / 58.38 FPS
Dense-scene motion with 1,500 temporarily drawn meshes	57.84 FPS
Restore all 259,800 meshes	429.01 ms
Restore including the quiet delay after motion	630.28 ms
Optimized full-detail draw, mean / 95th percentile	453.45 / 533.82 ms

8.2 Minimum configuration: what is established

The qualification target is continuous rotation at an actual 1920×1080 **render target and at least 30 FPS, with every one of the 259,800 stickers submitted on every frame and motion detail reduction disabled**. The frame budget is 33.33 ms. No tested GPU configuration in the retained evidence satisfies that target, so a minimum GPU model cannot yet be certified. Intel HD Graphics 620 is a validated compatibility baseline, but it is not a qualifying full-detail performance configuration.

The current rendering path retains CPU-side projection and triangle/line packing. During the eighteen optimized full-detail frames, total native-process CPU consumption averaged approximately 329.86 ms per frame. Process CPU counters are not a clean separation of serialized CPU work from GPU time, but they corroborate the source-level finding that this is not a GPU-only workload. Naming a discrete GPU from specification-sheet throughput would therefore be unsupported. At the smaller tested viewport, reaching the 30 FPS budget already requires about 13.60 times less elapsed time per complete frame. Moving to 1080p also increases pixel area by about 3.69 times; frame time should not be assumed to scale by that factor because geometry and CPU costs remain.

Table 5: Established compatibility and the limits of the full-detail hardware evidence.

Requirement	Evidence-backed status
Demonstrated native configuration	Windows 10 x64, x86 native host, installed .NET Framework 4.8, and official Managed DirectX 1.1 assemblies with a working Direct3D9 hardware driver
GPU demonstrated to render the complete model	Intel HD Graphics 620, on the tested Windows system
GPU demonstrated to sustain full-detail 1080p/30 FPS	None established
Minimum VRAM guaranteeing the target	Not established; capacity alone does not certify throughput
Minimum purchase specification	No defensible GPU-only minimum until the complete CPU/GPU/driver configuration is measured

The legacy DirectX components remain an external dependency; Microsoft documents the June 2010 end-user runtime distribution [10]. A newer DirectX feature level or a larger VRAM figure alone does not demonstrate that the required legacy runtime is installed or that the CPU-bound portions meet the frame budget.

8.3 Qualification protocol

A qualifying hardware test should report the exact CPU, GPU, driver, operating system, runtime, source/build hashes, render-target dimensions, and rendering settings. It should use the same full

model and complete sticker set, disable level-of-detail substitution, retain the same clipping and frame policy, and include continuous 3D and 4D rotations over fixed reproducible camera paths. A warmed run of at least sixty seconds should report frame-interval percentiles, sustained FPS, CPU use, memory use, and submitted mesh counts. A conservative pass requires sustained mean FPS of at least 30 and a 95th-percentile frame interval no greater than 33.33 ms, without missing geometry or state divergence.

Cold construction, full-detail restoration, picking latency, and stationary redraws should be measured separately. All labels must remain identical across the camera-only workload, and sampled full-detail images must agree with the established rendering reference. Until such a configuration passes, the practical tested option remains exact filtered work or adaptive motion. The open optimization problem is to reduce the complete rendering pipeline’s cost, potentially including batching or GPU-side projection, and then establish a hardware minimum by measurement.

9 Limitations and research directions

The retained cut profile and enumerations define the scope of the mathematics. The lower bound in Section 4 is supported by a finite model/controller audit; it does not rely on random scramble success as a proof of group reachability. The all-orbit method has explicit certificates and finite setup coverage, but the report does not claim a separately machine-checked formal proof in a theorem prover or a published exact group order.

The present seed bank emphasizes correctness over economical primitive words. In the seed-600 workload, O15, O29, and O02 account for roughly 40.3% of represented primitive moves while accounting for only about 3.8% of stars. Shorter witnesses for these stages, paired insertions, and separately certified alternative buffer pairs are concrete research targets. Replacing a seed requires rechecking its complete effect, orientation handling, dependency order, and full-state replay; a shorter word is not automatically a safe replacement [1, p. 31]; [2].

Empirical human-solving work should define its assistance policy before an attempt begins. Useful outcomes include measured recognition and target-selection time, error rates, recovery cost, and the relationship between local batching and attention. Such observations would test the transcript’s workload scenarios instead of treating them as predictions already confirmed by a solver.

The native validation is hardware-specific. Controlled device-loss and process-termination checks do not simulate every driver fault or physical power loss. The unattended tests exercise production restore/file-processing paths but do not claim manual coverage of every common file or confirmation dialog. Likewise, a verified full-state bridge does not certify every possible upstream binary as compatible. These boundaries allow the demonstrated results to be used without enlarging them into unsupported guarantees.

10 Conclusion

The full 600-cell becomes tractable as a computational control problem when its pieces are separated into actual move orbits, orientations are represented by their true groups, and every macro retains its legal witness and complete collateral action. The original design transcript develops that construction and the accompanying reference makes its thirty-five seeds and setup machinery explicit. The conservative color-state bound explains the extraordinary scale relative to a documented solved high-dimensional benchmark while avoiding a universal difficulty claim.

The subsequent workbench turns those ideas into a tested native workflow with coherent state, recoverable operations, exact filtering, and measurable performance. Its present strength is verified macro-assisted control and responsive filtered or adaptive interaction. Economical human

algorithms and continuous full-detail 1080p/30 FPS remain distinct research and engineering targets. The public report, original algorithm reference, synthetic replay companion, and bounded audit make these achievements and remaining questions independently inspectable.

Data availability and attribution

The report, its source, the unchanged 51-page algorithm reference, exact orbit words, selected generated tables, three synthetic compact solve records, and independent count audit accompany the C600-MPUlt repository. The companion introduction distinguishes historical reports from newly repeated checks. The original exported conversation is retained privately; its technical evidence is mapped by page range without publishing account identifiers or personal exchanges. Personal databases, screenshots, raw diagnostics, original third-party solve logs, and derived personal-history traces are excluded. No proprietary runtime assemblies are added with these research files.

Primary credit for the original MPUlt simulator and renderer belongs to Andrey Astrelin. The C600 workbench is a mod and retains its upstream notices. Nan Ma’s solving methods and ivan216’s software projects are acknowledged as antecedents. The source corpus includes AI-assisted analysis and code generation; its assertions are evaluated through explicit arguments, retained certificates, and stated execution checks rather than by treating conversational output as an authority. This technical report is not presented as a peer-reviewed publication.

References

- [1] Project archive. *600 Cell Solving Strategy*. Exported design transcript, 52 pages, 2026. Privately retained primary documentary source; public page-range guide supplied with this report.
- [2] Project archive. *Full 600-Cell: 35-Orbit Algorithms*. Computational algorithm reference, 51 pages, 2026. Original supplied PDF and exact-word companion retained with the public research materials.
- [3] Astrelin, A. Original announcement of the full 600-cell, 27 May 2011. Hypercubing archive, message 1743. <https://archive.hypercubing.xyz/messages/1743.html> Original MPUlt project and authorship: <https://superliminal.com/andrey/mpu/>
- [4] Ma, N. Detailed simplified 600-cell solving method, 2012. Hypercubing archive, message 2443. <https://archive.hypercubing.xyz/messages/2443.html>
- [5] Astrelin, A. *MC7D: Hall of Fame and puzzle documentation*. Original project site; accessed 7 September 2026. <https://superliminal.com/andrey/mc7d/>
- [6] Gottlieb, M., and other participants. Completion account for the team $256 \times 256 \times 256$ solve, 2021. Speedsolving discussion, page 4, especially post 67. <https://www.speedsolving.com/threads/256x256-rubiks-cube-solve-world-record-attempt.79808/page-4>
- [7] C600 Studio contributors. *0.2.4 Release Validation and MPUlt Runtime Provenance*. C600-MPUlt repository, 2026. <https://github.com/KonomiYuzu01/C600-MPUlt/blob/main/tests/VALIDATION.md> and https://github.com/KonomiYuzu01/C600-MPUlt/blob/main/docs/RUNTIME_PROVENANCE.md
- [8] ivan216. *MPUlt*. Source project. <https://github.com/ivan216/MPUlt>
- [9] ivan216. *mc7d-kb*. Source project. <https://github.com/ivan216/mc7d-kb>
- [10] Microsoft. *DirectX End-User Runtimes (June 2010)*. Official runtime distribution. <https://www.microsoft.com/en-us/download/details.aspx?id=8109>